

CMMS

1. Opis Aplikacji

Główne funkcjonalności

2. Opis podziału pracy

Patryk Stafecki

Jakub Owczarzak

Piotr Krzeszowski

3. Schematy bazy danych

Baza relacyjna - MariaDB

Lokalna baza danych (Room)

4. Schemat nawigacji

Dostęp do akcji według roli

5. Instrukcje uruchomienia

Backend

Zatrzymanie aplikacji

Aplikacja mobilna

1. Opis Aplikacji

CMMS Mobile to mobilna aplikacja Android wspomagająca zarządzanie utrzymaniem ruchu maszyn (Computerized Maintenance Management System). Aplikacja stanowi mobilne rozszerzenie systemu webowego i umożliwia pracownikom zakładu zarządzanie zleceniami serwisowymi, maszynami oraz powiadomieniami bezpośrednio z urządzenia mobilnego.

Aplikacja obsługuje cztery role użytkowników:

- **Administrator** — pełny dostęp do wszystkich funkcji systemu
- **Menedżer** — zarządzanie zleceniami i maszynami
- **Technik** — podgląd i realizacja przypisanych zleceń serwisowych
- **Operator** — zgłaszanie awarii i śledzenie statusu własnych zgłoszeń

Główne funkcjonalności

- Logowanie z autoryzacją JWT i automatycznym odświeżaniem sesji
- Dashboard z podsumowaniem otwartych zleceń, awarii krytycznych i zaplanowanych przeglądów
- Lista maszyn z możliwością wyszukiwania, dodawania i edycji
- Lista zleceń serwisowych z filtrowaniem po statusie, przypisywaniem techników i zarządzaniem częściami
- Fragment powiadomień z oznaczaniem jako przeczytane
- Lokalne powiadomienia push dla nowych nieodczytanych powiadomień
- Tryb offline — dane przechowywane lokalnie w bazie Room, baner informujący o braku połączenia
- Profil użytkownika z listą certyfikatów i wylogowaniem

2. Opis podziału pracy

Projekt realizowany był przez trzech członków zespołu:

Patryk Stafecki

- Napisanie całego backendu aplikacji (API REST, baza danych, logika biznesowa)
- Inicjalizacja projektu mobilnego i konfiguracja struktury
- Warstwa sieciowa — klient Retrofit, modele odpowiedzi API
- Baza danych Room — encje, DAO, AppDatabase
- Warstwa repozytoriów — AuthRepository, MachineRepository, WorkOrderRepository, NotificationRepository
- Warstwa ViewModels ze wzorcem MediatorLiveData dla wszystkich ekranów
- Obsługa trybu offline — lokalny fallback danych, blokada edycji bez połączenia, baner offline
- Zarządzanie sesjami — odświeżanie tokenów, obsługa wygaśnięcia sesji
- Zarządzanie technikami i częściami w zleceniach

Jakub Owczarzak

- Ekran i nawigacja — struktura fragmentów z NavController i dolnym paskiem nawigacji
- UI wszystkich fragmentów — Dashboard, Lista maszyn, Szczegóły maszyny, Lista zleceń, Szczegóły zlecenia, Powiadomienia, Profil
- Motywy i stylizacja aplikacji
- Zarządzanie tokenem JWT w SharedPreferences i mechanizm auto-logowania
- Podłączenie fragmentów do nowych ViewModels i usunięcie starych
- Lokalne powiadomienia push (NotificationHelper, kanały powiadomień, uprawnienie runtime)
- Naprawa błędu NullPointerException przy edycji maszyny

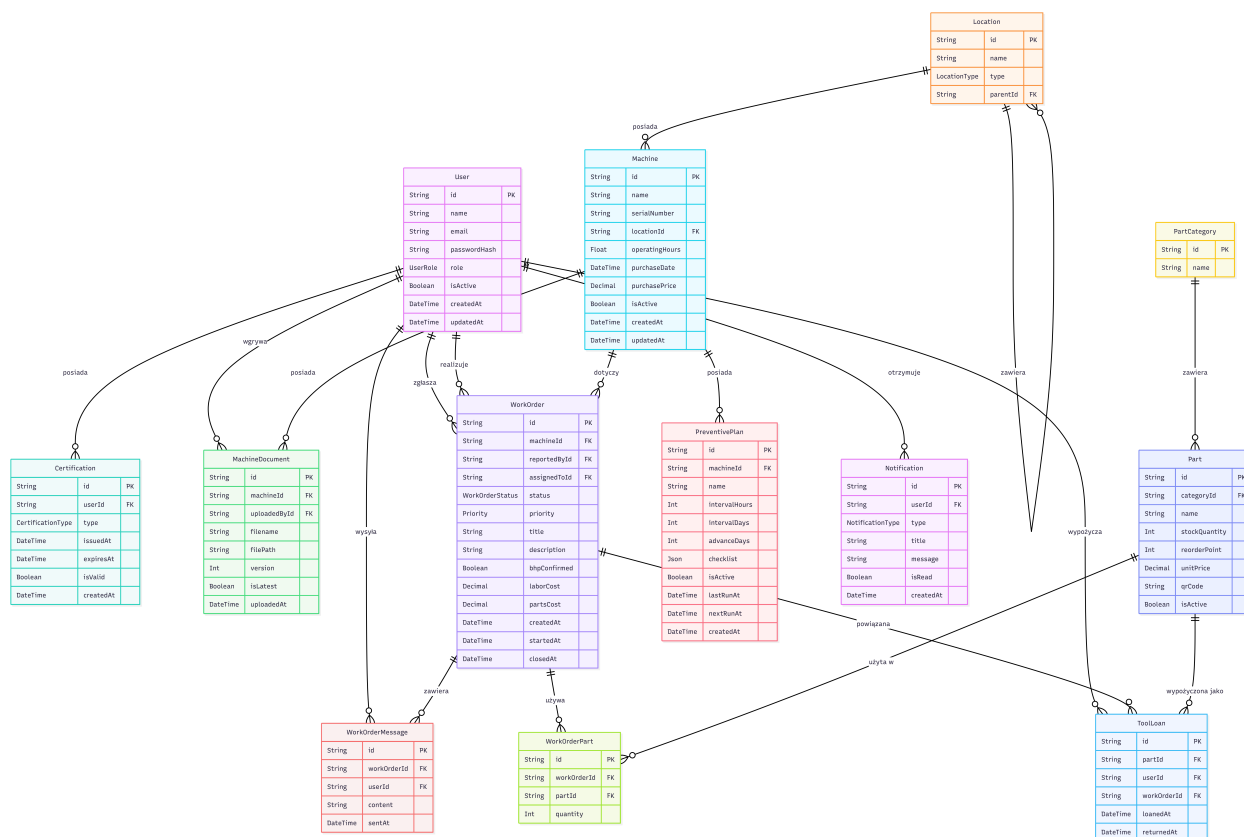
Piotr Krzeszowski

- Pierwsze wersje ekranów logowania, dashboardu, listy maszyn i szczegółów maszyny
- Aktualizacja ikony aplikacji

3. Schematy bazy danych

Baza relacyjna - MariaDB

Główna baza danych przechowuje wszystkie dane biznesowe aplikacji. Schemat zdefiniowany jest w Prisma (`schema.prisma`) i podzielony na 6 modułów tematycznych.



Lokalna baza danych (Room)

Aplikacja wykorzystuje lokalną bazę danych Room jako cache danych z API, umożliwiając działanie w trybie offline. Dane są synchronizowane z backendem przy każdym odświeżeniu widoku.

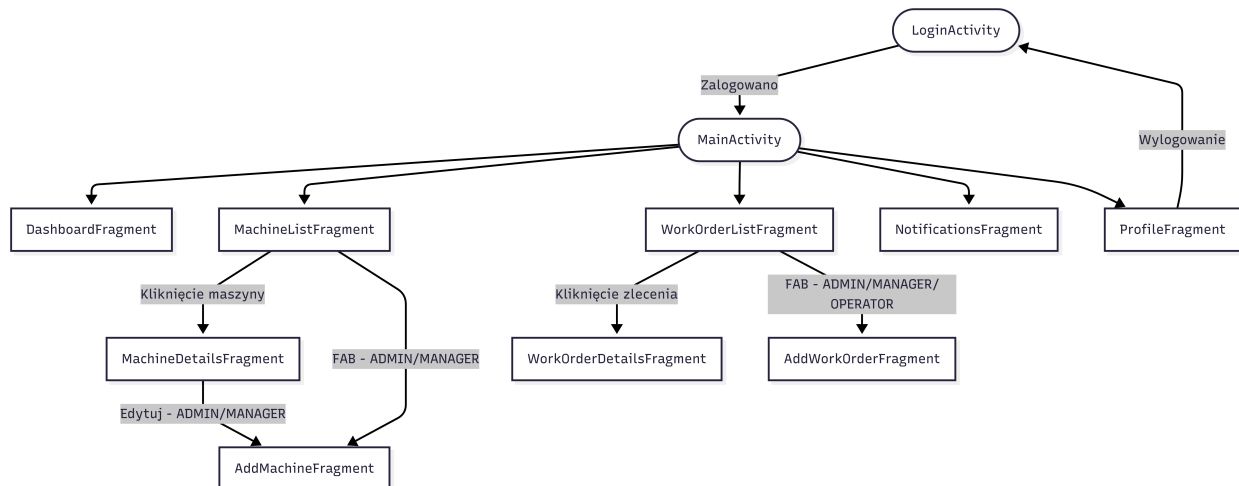
machines		
String	id	PK
String	name	
String	serialNumber	
double	operatingHours	
boolean	isActive	
String	locationId	
String	locationName	
String	purchaseDate	
double	purchasePrice	



work_orders		
String	id	PK
String	title	
String	status	
String	priority	
String	description	
boolean	bhpConfirmed	
String	createdAt	
String	machineId	FK
String	machineName	
String	assignedToId	
String	assignedToName	
String	reportedById	
String	reportedByName	
String	partsText	

notifications		
String	id	PK
String	type	
String	title	
String	message	
boolean	isRead	
String	createdAt	

4. Schemat nawigacji



Dostęp do akcji według roli

Akcja	ADMIN	MANAGER	TECHNICIAN	OPERATOR
Dodaj maszynę	✓	✓	—	—
Edytuj maszynę	✓	✓	—	—
Dodaj zlecenie	✓	✓	—	✓
Edytuj zlecenie	✓	✓	—	—
Zmień status zlecenia	✓	✓	✓	—
Widok wszystkich zleceń	✓	✓	—	—

5. Instrukcje uruchomienia

Backend

- Wymagania:

Docker Desktop, Node.js 22.x

Zacynamy od sklonowania repozytorium zawierającego api oraz instalacji zależności:

```
git clone https://github.com/stafecki/cmms.git
cd cmms
npm install
```

Skopiuj i uzupełnij zmienne środowiskowe

```
cp .env.example .env
```

Polecenie buduje i uruchamia wszystkie kontenery:

```
npm run docker:up
```

Uruchom migracje:

```
npm run docker:migrate
```

Załaduj dane testowe:

```
npm run docker:seed
```

Domyślne konto administratora po seedowaniu bazy danych dostępne jest w pliku `apps/api/prisma/seed.ts`.

Serwer domyślnie działa na `http://localhost:3000`.

Zatrzymanie aplikacji

```
npm run docker:down
```

Aplikacja mobilna

Wymagania: Android Studio, Android SDK, urządzenie fizyczne lub emulator (min. Android 8.0 API 26)

1. Otwórz folder `app/` w Android Studio

2. Upewnij się że backend działa lokalnie
3. W pliku `app/src/main/java/com/example/cmms/data/remote/ApiClient.java` sprawdź adres bazowy API:
 - Emulator: `http://10.0.2.2:3000/`
 - Urządzenie fizyczne: `http://<IP-komputera>:3000/`
4. Kliknij **Run** lub `Shift+F10`